

NAME: TANKISO MASOEBE

COMPUTER ARCHITECTURE WK6

1. Scenario Analysis: RISC/CISC in an IoT Device

An IoT device such as a smart home temperature sensor needs a processor that can perform tasks efficiently while using very little power. A **RISC (Reduced Instruction Set Computer)** processor is suitable for this scenario because it uses a small set of simple instructions that can be executed quickly. For example, an ARM-based RISC processor can read temperature data from sensors, process the information, and send it to a mobile application or cloud server. Its low power consumption helps the device operate for a long time using a battery. In contrast, a **CISC (Complex Instruction Set Computer)** processor uses more complex instructions that can perform several operations in one instruction. CISC processors can be useful when an IoT device requires more complex processing. However, they may consume more power. Therefore, RISC is commonly suitable for small IoT devices because it provides efficient processing, low power consumption, and reduced hardware complexity.

2. Concept Research: RISC Load-Store Architecture

RISC (Reduced Instruction Set Computer) load-store architecture is a processor design where only load and store instructions access memory. Load instructions transfer data from memory into registers, while store instructions transfer data from registers into memory. Other instructions, such as arithmetic and logical operations, work only on data stored in registers. This design simplifies the processor's instruction set and allows instructions to execute efficiently. RISC processors commonly use fixed-length instructions and a large number of registers. Load-store architecture improves processing speed, reduces hardware complexity, and supports efficient pipelining. It is widely used in ARM processors found in smartphones, embedded systems, and IoT devices because of its performance and low power consumption.

3. Tool Practice: Simulating a CISC Instruction in an x86 Emulator

I simulated a CISC instruction using an x86 emulator to understand how complex instructions are executed by a processor. The instruction used was MOV, which transfers data between registers and memory. The emulator showed how the instruction is decoded and executed using the CPU registers and memory. I observed that x86 instructions can have different lengths and support multiple addressing modes, making the instruction set flexible. The simulation helped me understand how CISC processors can perform complex operations using fewer instructions. It also showed that this flexibility can increase processor complexity. Overall, the practical exercise improved my

understanding of CISC architecture and how x86 processors execute instructions at the hardware level.

4. DIAGRAM:

